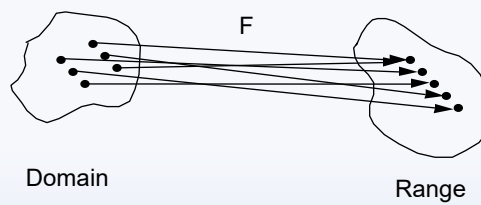


Equivalence Class Testing

1

Equivalence Class Testing

- Equivalence class testing uses information about the functional mapping itself to identify test cases



2

Equivalence Relations

- Given a relation R defined on some set S , R is an equivalence relation if (and only if), for all x , y , and z elements of S :
 - R is reflexive, i.e., xRx
 - R is symmetric, i.e., if xRy , then yRx
 - R is transitive, i.e., if xRy and yRz , then xRz
- An equivalence relation, R , induces a partition on the set S , where a partition is a set of subsets of S such that:
 - The intersection of any two subsets is empty, and
 - The union of all the subsets is the original set S
- **Note:** the intersection property assures no redundancy, and the union property assures no gaps.

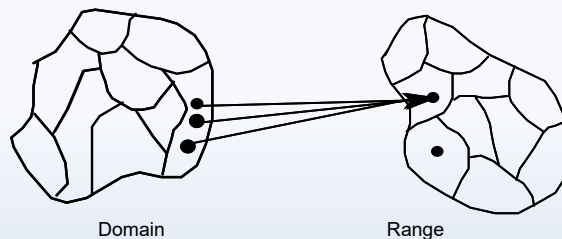
3

Equivalence Partitions

Define a relation R on the input domain D as:

$$\forall x, y \in D, xRy \text{ iff } F(x) = F(y),$$

where F is the program function.



Equivalence Relations (e.g., R) treat inputs the *same*.

4

Equivalence Partitioning

- Works best when F is a many-to-one function
- Test cases are formed by selecting one value from each equivalence class.
- Identifying the classes may be hard

5

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- **Normal**: classes of valid values of inputs
- **Robust**: classes of valid and invalid values of inputs
- **Weak**: (single fault assumption) one from each class
- **Strong**: (multiple fault assumption) one from each class in Cartesian Product

6

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- **Normal**: classes of valid values of inputs
- **Robust**: classes of valid and invalid values of inputs
- **Weak**: (single fault assumption) one from each class
- **Strong**: (multiple fault assumption) one from each class in Cartesian Product

7

“Traditional” Equivalence Class Testing

- Programmer arrogance:
 - In the 1960s and 1970s, programmers often had very detailed input data requirements.
 - If input data didn’t comply, it was the user’s fault
 - The popular phrase—Garbage In, Garbage Out (GIGO)
- Programs from this era soon developed defenses
 - (many of these programs are STILL legacy software)
 - as much as 75% of code verified input formats and values
- “Traditional” equivalence class testing focuses on detecting invalid input.
 - (almost the same as our “weak robust equivalence class testing”)

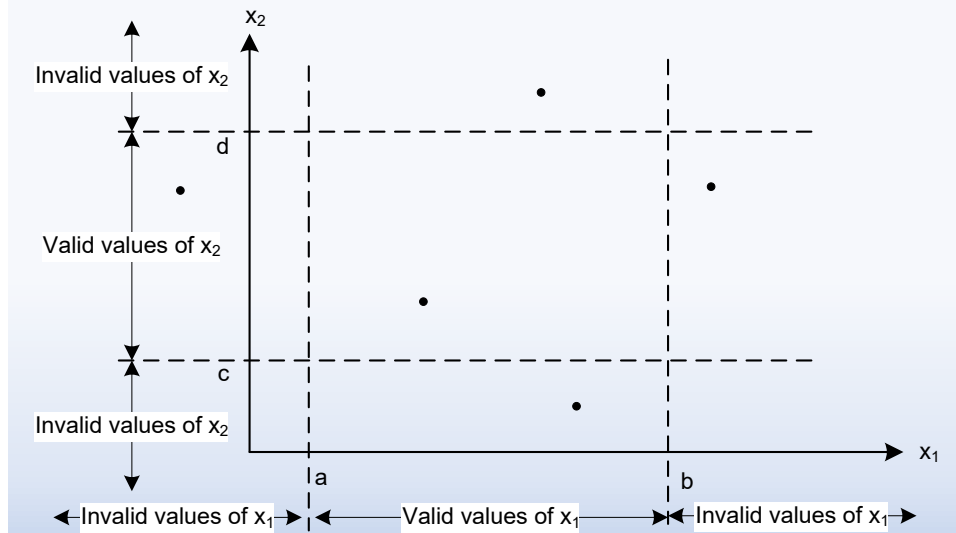
8

Example Inputs

- **F(x1, x2) has these classes:**
 - valid values of x1: $a \leq x_1 \leq b$
 - invalid values of x1: $x_1 < a, b < x_1$
 - valid values of x2: $c \leq x_2 \leq d$
 - invalid values of x2: $x_2 < c, d < x_2$
- **Process**
 1. Test F for valid values of all variables,
 2. Then test one invalid variable at a time

9

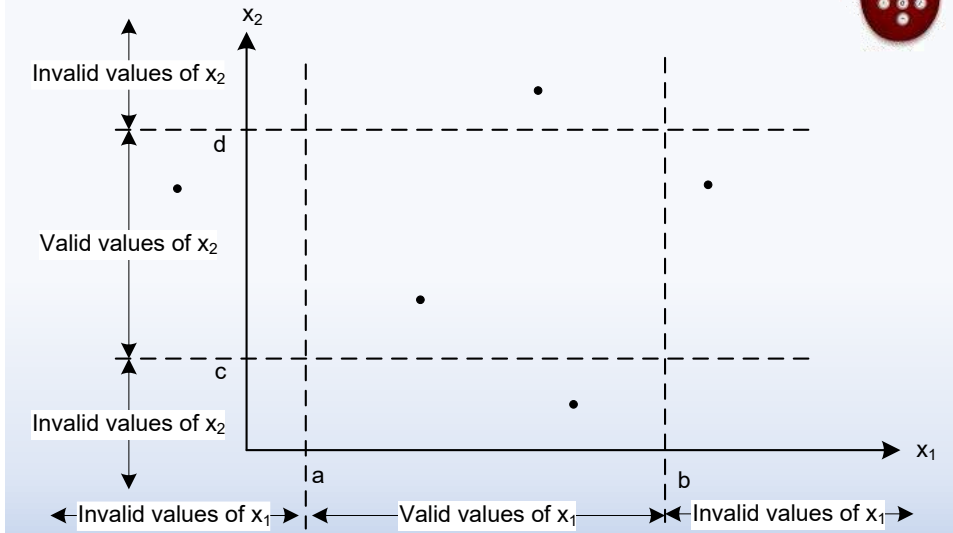
Traditional Equivalence Class Testing



10

In-Class Question

These tests make the single-fault assumption. (True or False)



11

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- Attributes

{

 - **Normal**: classes of valid values of inputs
 - **Robust**: classes of valid and invalid values of inputs
 - **Weak**: (single fault assumption) one from each class
 - **Strong**: (multiple fault assumption) one from each class in Cartesian
- Product

12

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- Attributes

{

 - **Normal**: classes of valid values of inputs
 - **Robust**: classes of valid and invalid values of inputs
 - **Weak**: (single fault assumption) one from each class
 - **Strong**: (multiple fault assumption) one from each class in Cartesian Product

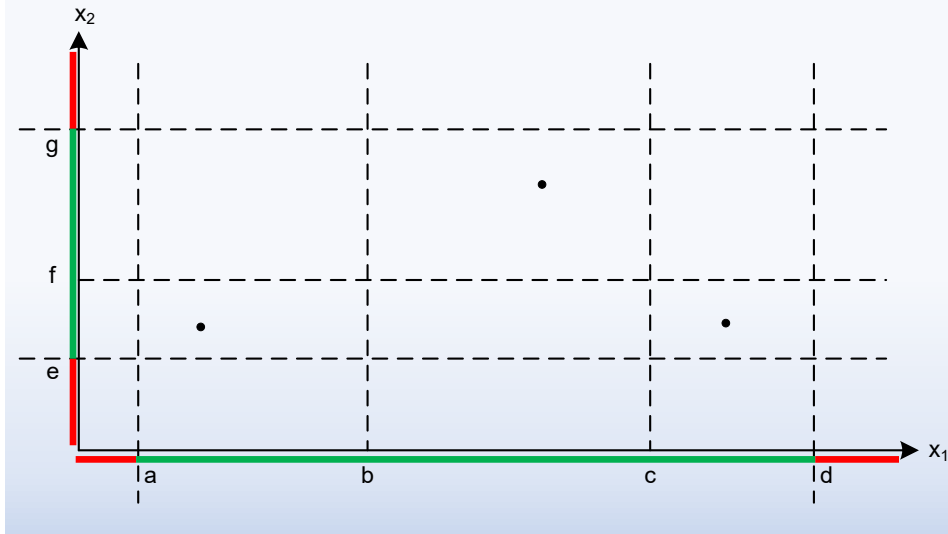
13

Weak Normal Equivalence Class Testing

- Identify equivalence classes of valid values.
- Test cases have all valid values.
- Detects faults due to calculations with valid values of a single variable.
- OK for regression testing.
- Need an expanded input example to illustrate:
 - **valid classes**: { $a \leq x_1 < b$ }, { $b \leq x_1 < c$ }, { $c \leq x_1 \leq d$ }, { $e \leq x_2 < f$ }, { $f \leq x_2 \leq g$ }
 - **invalid classes**: { $x_1 < a$ }, { $x_1 > d$ }, { $x_2 < e$ }, { $x_2 > g$ }

14

Weak Normal Equivalence Class Test Cases



15

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- Attributes {
 - **Normal**: classes of valid values of inputs
 - **Robust**: classes of valid and invalid values of inputs
 - **Weak**: (single fault assumption) one from each class
 - **Strong**: (multiple fault assumption) one from each class in Cartesian Product

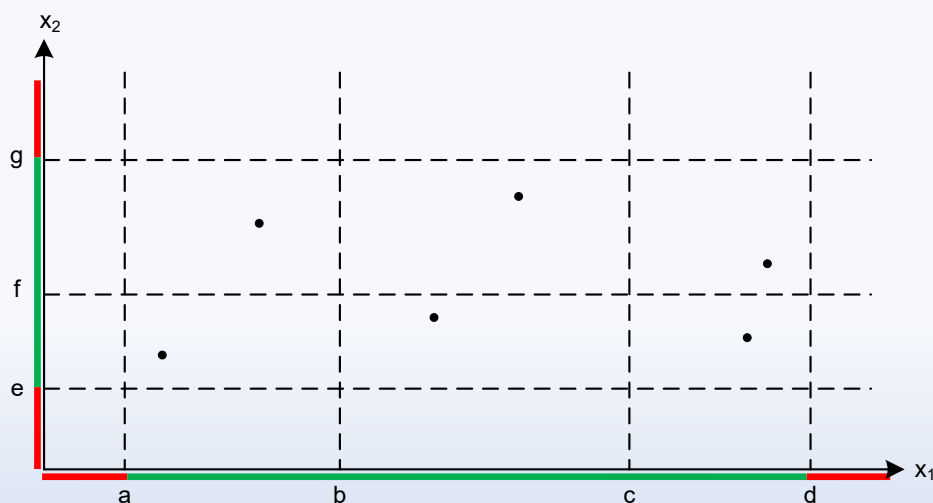
16

Strong Normal Equivalence Class Testing

- Identify equivalence classes of valid values.
- Test cases have all valid values.
- Detects faults due to interactions with any values of any number of variables
- OK for regression testing.

17

Strong Normal Equivalence Class Test Cases



18

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- Attributes

{

 - **Normal**: classes of valid values of inputs
 - **Robust**: classes of valid and invalid values of inputs
 - **Weak**: (single fault assumption) one from each class
 - **Strong**: (multiple fault assumption) one from each class in Cartesian Product

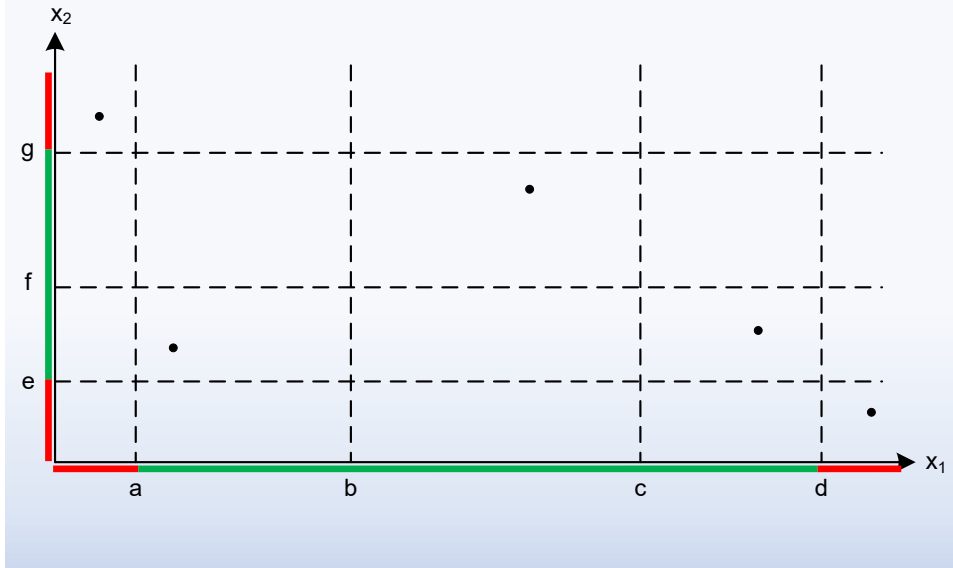
19

Weak Robust Equivalence Class Testing

- Identify equivalence classes of valid and invalid values.
- Test cases have all valid values except one invalid value.
- Detects faults due to calculations with valid values of a single variable.
- Detects faults due to invalid values of a single variable.
- OK for regression testing.

20

Weak Robust Equivalence Class Test Cases



21

Forms of Equivalence Class Testing

- **“Traditional”**: focus on invalid inputs
- Attributes {
 - **Normal**: classes of valid values of inputs
 - **Robust**: classes of valid and invalid values of inputs
 - **Weak**: (single fault assumption) one from each class
 - **Strong**: (multiple fault assumption) one from each class in Cartesian Product

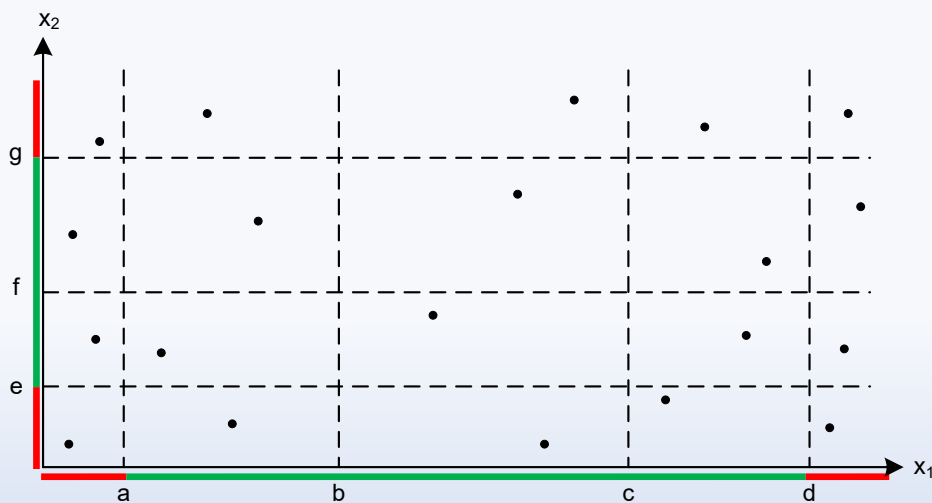
22

Strong Robust Equivalence Class Testing

- Identify equivalence classes of valid and invalid values.
- Test cases from Cartesian Product of all classes.
- Detects faults due to interactions with any values of any number of variables.
- OK for regression testing, better for progression testing.
 - Most rigorous form of Equivalence Class testing but the number of test cases grow *quickly*!

23

Strong Robust Equivalence Class Test Cases



24

NextDate Example

NextDate is a function of three variables: month, date, and year. It returns the date of the day after the input date. The month, date, and year variables have integer values subject to these conditions:

- c1. $1 \leq \text{month} \leq 12$
- c2. $1 \leq \text{day} \leq 31$
- c3. $1812 \leq \text{year} \leq 2112$

If any of conditions c1, c2, or c3 fails, NextDate produces an output indicating the corresponding variable has an out-of-range value — for example, “Value of month not in the range 1..12”. Because numerous invalid day–month–year combinations exist, NextDate collapses these into one message: “Invalid Input Date.”

25

In Class Assignment!

- Identify equivalence classes for the NextDate example!
- Your solution should call out
 - the specification variables,
 - Ranges of those variables that are equivalence classes

NextDate is a function of three variables: month, date, and year. It returns the date of the day after the input date. The month, date, and year variables have integer values subject to these conditions:

- c1. $1 \leq \text{month} \leq 12$
- c2. $1 \leq \text{day} \leq 31$
- c3. $1812 \leq \text{year} \leq 2112$

26

Selecting an Equivalence Relation

If x and y are days, *some possibilities* for NextDate are:

- $x R y$ iff x and y are mapped onto the same year
- $x R y$ iff x and y are mapped onto the same month
- $x R y$ iff x and y are mapped onto the same date
- $x R y$ iff $x(\text{day})$ and $y(\text{day})$ are “treated the same”
- $x R y$ iff $x(\text{month})$ and $y(\text{month})$ are “treated the same”
- $x R y$ iff $x(\text{year})$ and $y(\text{year})$ are “treated the same”

There is no such thing as **THE** equivalence relation.

Best practice is to select an **equivalence relation** that **reflects the behavior being tested**.

27

NextDate Equivalence Classes

- **Month:**

- $M1 = \{ \text{month} : \text{month has 30 days} \}$
- $M2 = \{ \text{month} : \text{month has 31 days} \}$
- $M3 = \{ \text{month} : \text{month is February} \}$

- **Day:**

- $D1 = \{ \text{day} : 1 \leq \text{day} \leq 28 \}$
- $D2 = \{ \text{day} : \text{day} = 29 \}$
- $D3 = \{ \text{day} : \text{day} = 30 \}$
- $D4 = \{ \text{day} : \text{day} = 31 \}$

- **Year:**

- $Y1 = \{ \text{year} : \text{year is a leap year} \}$
- $Y2 = \{ \text{year} : \text{year is not a leap year (common year)} \}$

28

Discussion

When would you use Equivalence Class Testing?

29

When to Use Equivalence Class Testing

- Variables represent logical (rather than physical) quantities.
- Variables “support” useful equivalence classes.
- Distinct modes or sets of discrete behavior.

30

How to choose an Equivalence Class method?

	Valid Values	Valid and Invalid Values
Single Fault	Weak Normal	Weak Robust
Multiple Fault	Strong Normal	Strong Robust